

ALBUKHARY INTERNATIONAL UNIVERSITY
CCC2143 SOFTWARE ENGINEERING — GROUP PROJECT REPORT

ScholarFit

An AI-Powered Scholarship Matching Platform

Software Engineering Requirements Report

Project Category: Education-based System

Course Code	CCC2143	Programme	BCS (Bachelor of Computer Science)
Course Title	Software Engineering	Academic Session	2025/2026
Classification	Core Computing	Semester	3
Lecturer	Nadiah Arsat	Assessment Method	Group Project (20%)
Project Category	Education-based System	Submission Deadline	16 September 2026

Prepared by (Group Members):

1. Malitos — [Matric No.]
2. [Group Member Name] — [Matric No.]
3. [Group Member Name] — [Matric No.]
4. [Group Member Name] — [Matric No.]

Submitted to: Nadiah Arsat

Date Prepared: 17 August 2026

Citation Style: APA 7th Edition

Table of Contents

1. Introduction.....	3
2. Project Objectives.....	4
2.1 Objectives Related to Acceptability and Efficiency.....	4
2.2 Objectives Related to Dependability and Security.....	4
2.3 Objectives Related to Maintainability and Efficiency.....	4
2.4 Objectives Related to Documentation and Communication.....	5
2.5 Measurable Success Criteria.....	5
3. System Requirements Analysis.....	7
3.1 Requirements Engineering Process.....	7
3.2 System Actors.....	7
3.3 Use Case Summary.....	7
3.4 Detailed Use-Case Specification — UC-05.....	8
3.5 Functional Requirements.....	9
3.6 Non-Functional Requirements.....	11
4. System Models.....	13
4.1 Activity Diagram.....	13
4.2 Sequence Diagram.....	14
4.3 Class Diagram.....	14
4.4 State Diagram.....	16
5. System Architecture Design.....	18
5.1 Architectural Pattern Selection and Justification.....	18
5.2 Architecture Diagram.....	19
5.3 Supporting Design Decisions.....	20
6. Component-Based Model.....	21
6.1 Component Catalogue.....	21
6.2 Component Diagram.....	22
6.3 Composition Types.....	22
6.4 Reusability Characteristics.....	23
7. System Testing Strategy.....	24
7.1 Testing Objectives and Expected Outcomes.....	24
7.2 Testing Pipeline.....	24
7.3 Testing Techniques.....	25
7.4 Testing the Non-Deterministic AI Pipeline.....	26
7.5 Sample Test Case Documentation.....	26
7.6 Quality Attributes Targeted by the Test Strategy.....	27
7.7 Acceptance Testing Process.....	27
8. Gantt Chart.....	28
References.....	29

1. Introduction

Software engineering is an engineering discipline that is concerned with all aspects of software production, from the early stages of system specification through to maintaining the system after it has entered use (Sommerville, 2016). As an engineering discipline, it requires the team to apply theories, methods and tools selectively and appropriately, while working within organisational and financial constraints; as a practice concerned with all aspects of software production, it covers not only technical development but also project management, requirements analysis, system modelling, architectural design and testing strategy. This report applies that discipline to a single case: ScholarFit, an AI-powered scholarship-matching platform proposed under the Education-based System category of the CCC2143 Software Engineering group project.

Prospective scholarship applicants spend the majority of their effort — roughly 70% of their time, by the project sponsor's estimate — searching for scholarships they might qualify for, and only about 20% of their time on the application itself. This imbalance exists because eligibility criteria vary widely across providers (nationality, current location, education level, prior institution, qualifications, passport status, required documents, deadlines), and no existing platform verifies a student's identity and academic standing once and then reuses that verification across every scholarship in its catalogue. Students therefore repeatedly assemble the same documents for opportunities they may not even qualify for, while scholarship providers and partner organisations have no reliable channel to reach a pool of pre-qualified, verified candidates.

The proposed solution can be summarised using a product vision statement:

FOR international and domestic students actively searching for higher-education scholarships, WHO waste most of their effort discovering which opportunities they are even eligible for, THE ScholarFit platform IS an AI-powered verification-and-matching system THAT verifies a student's identity, academic profile and supporting documents once, and continuously matches the verified profile only against scholarships the student genuinely qualifies for. UNLIKE generic scholarship directories and listing sites that display every opportunity regardless of eligibility, OUR PRODUCT inverts the effort ratio — from roughly 70% searching / 20% applying to the reverse — by combining an AI document-verification engine with a rule-based eligibility-matching engine, backed throughout by a human-in-the-loop admin review path for cases the AI cannot confidently decide.

ScholarFit is built around two user roles — Student and Admin — and a core flow in which a student registers, completes a detailed profile and uploads supporting documents (transcripts, passport/ID, proof of graduation), submits the profile for review, and receives either automatic AI verification or a routed admin decision together with an explanatory email. Once verified, the student is shown only the scholarships their profile matches, sourced from both manually entered records and third-party partner feeds, and can apply directly through the platform. This places ScholarFit squarely in the Education-based System category of the assignment brief, since its entire purpose is to widen and simplify students' access to educational funding.

This report documents the requirements analysis, system modelling, architectural design, component-based design and testing strategy for ScholarFit, in that order, following the structure prescribed in the CCC2143 assessment instructions. Section 2 sets out the project's objectives; Section 3 analyses the system's functional and non-functional requirements; Section 4 presents the four required UML system models (Activity, Sequence, Class and State diagrams); Section 5 proposes and justifies a system architecture; Section 6 develops a component-based model of the system; Section 7 defines a testing strategy appropriate to a system with a non-deterministic AI core; and Section 8 presents a Gantt chart for the proposed development schedule.

2. Project Objectives

Sommerville (2016) identifies four essential attributes that any well-engineered software product should exhibit: maintainability, dependability and security, efficiency, and acceptability. The objectives of the ScholarFit project are framed directly against these four attributes, so that every objective is traceable to a recognised software quality criterion rather than to a vague notion of “good practice”.

2.1 Objectives Related to Acceptability and Efficiency

- **O1 — Single-verification profile.** To design a profile and document-verification model in which a student's identity and academic credentials are verified once and reused across every scholarship match, removing the need to re-submit the same documents repeatedly.
- **O2 — Eligibility-driven matching.** To design an AI/rule-based matching engine that evaluates a verified student's profile against every scholarship's structured eligibility rules and surfaces only the opportunities the student genuinely qualifies for, directly reducing the ~70% of effort students currently spend searching.
- **O3 — Usable, role-appropriate dashboards.** To design a Student Dashboard and an Admin Dashboard that are understandable and usable by their respective, non-technical and technical audiences, and that are compatible with the browsers and devices students and administrators already use (acceptability).

2.2 Objectives Related to Dependability and Security

- **O4 — Reliable AI-assisted verification with human oversight.** To design a verification pipeline in which the AI verification engine auto-verifies only high-confidence cases and escalates every ambiguous, inconsistent or low-confidence case to an Admin, so that no unverified or fraudulent account can reach the matching stage undetected.
- **O5 — Secure handling of sensitive documents.** To design a system architecture and data model that protects passports, national IDs, transcripts and other personally identifiable information through access control, encryption and audit logging of every verification decision.
- **O6 — Explainable outcomes.** To ensure that every rejection or escalation produced by the AI verification engine carries a specific, human-readable reason (e.g. “passport expiry page missing”) rather than a generic failure message, so that both students and admins can trust and act on the system's decisions.

2.3 Objectives Related to Maintainability and Efficiency

- **O7 — Layered, component-based architecture.** To design a layered system architecture, decomposed into independent, replaceable components with well-defined provided/required interfaces, so that the AI verification model, matching model or scholarship-sourcing strategy can each evolve independently as the platform grows (Sprint-based incremental delivery, Section 8).
- **O8 — Efficient use of AI resources.** To design a verification and matching pipeline that minimises unnecessary calls to external AI/OCR services (through a confidence-threshold gate and background job queue) so that the system remains responsive and cost-efficient as the number of students and scholarships grows.

2.4 Objectives Related to Documentation and Communication

- **O9 — Precise system models.** To produce Activity, Sequence, Class and State diagrams (Section 4) that are precise and complete enough for a development team to begin implementation without needing to consult the project sponsor for basic clarification, in line with the course's own worked examples.
- **O10 — A verifiable testing strategy.** To define a testing strategy (Section 7) that explicitly accounts for the non-deterministic nature of AI-generated verification and matching outcomes, rather than assuming conventional deterministic pass/fail testing is sufficient.

2.5 Measurable Success Criteria

So that each objective can be assessed rather than merely asserted, every objective above is restated below against a measurable success criterion and the requirement that carries it. The metric column is the evidence that would demonstrate the objective has been met; each traces forward to a numbered functional or non-functional requirement in Section 3 and, where applicable, to a testing objective in Section 7.1.

Objective	Measurable Success Criterion	Traces To
O1 — Single-verification profile	A verified student is never asked to re-upload an already-verified document for a subsequent scholarship match: zero repeat-upload prompts across all matches for a verified profile.	FR-03, FR-04, FR-06; TO-1
O2 — Eligibility-driven matching	100% of scholarships shown to a student satisfy every eligibility rule attached to them (zero ineligible results surfaced), measured against a test catalogue with known-correct expected matches.	FR-14, FR-15; TO-3
O3 — Usable, role-appropriate dashboards	A first-time student completes registration and profile submission unaided in under 20 minutes, in a single session.	NFR-03, NFR-06; TO-6
O4 — Reliable AI-assisted verification with human oversight	Every profile scoring below the auto-verify threshold reaches an Admin queue (zero silent auto-rejections), and the auto-verify false-positive rate stays under 1%.	FR-08, FR-09, NFR-04; TO-4
O5 — Secure handling of sensitive documents	No student or unauthorised admin can retrieve another student's document or verification record in access-control testing; 100% of Admin verification decisions appear in the audit log.	NFR-05, NFR-11, FR-12; TO-5
O6 — Explainable outcomes	100% of rejected or escalated verifications carry a specific reason string drawn from the defined reason set — zero generic "verification failed" messages.	FR-10, FR-11, NFR-10; TO-1
O7 — Layered, component-based architecture	Each of the eight components in Section 6 can be tested and replaced through its declared interface alone, with no test reaching into another component's internals.	Section 5, Section 6; TO-2
O8 — Efficient use of AI resources	95% of verification decisions are produced within 24 hours of submission, and no AI call blocks the request/response cycle of the web API.	NFR-01, NFR-07; TO-6
O9 — Precise system models	All four required UML models are present, internally consistent with one another, and each traceable to specific numbered requirements (demonstrated in	Section 4; TO-3

Objective	Measurable Success Criterion	Traces To
	Sections 4.1-4.4).	
O10 — A verifiable testing strategy	Every testing objective TO-1 to TO-7 has a stated method and a stated expected outcome that can be checked before release.	Section 7.1

3. System Requirements Analysis

3.1 Requirements Engineering Process

Requirements for ScholarFit were developed following the four requirements engineering sub-activities taught in the course: a feasibility study, requirements elicitation and analysis, requirements specification, and requirements validation (Sommerville, 2016). The feasibility study concluded that the project is technically feasible on the proposed stack (React/TypeScript, Bun, Hono, OpenAI API, PostgreSQL) and economically justified because the AI verification and matching pipeline can start on free/low-cost OCR and API tiers before any paid KYC vendor is required.

Elicitation and analysis combined three techniques taught in the course: (1) analysis of the project sponsor's product specification and problem statement (equivalent to a structured stakeholder interview); (2) scenario writing — for example, walking through "a Malaysian student uploads a blurry passport photo" to surface the need for an explainable-rejection requirement; and (3) use-case analysis to identify the discrete interactions between the Student, Admin and AI actors. Requirements were then specified using the user-requirement / system-requirement split and the "the system shall..." statement style used throughout the course's own Requirements Analysis chapter, and validated by checking each requirement against the criteria of validity, consistency, completeness, realism and verifiability.

3.2 System Actors

Actor	Type	Description
Student	Primary human actor	Registers, completes a profile, uploads documents, tracks verification status, views matches, applies to scholarships.
Admin	Primary human actor	Reviews AI-flagged accounts, manages scholarships and eligibility rules, monitors platform analytics.
AI Verification Engine	System actor	Performs OCR extraction and AI consistency/completeness checks on submitted profiles and documents.
AI Matching Engine	System actor	Evaluates a verified profile against every scholarship's eligibility rules and produces ranked matches.
Scholarship Partner	External actor	Supplies scholarship data through a third-party feed that is ingested by the Scholarship component.
Email Service	External actor	Delivers verification, rejection, match and deadline notifications to students.

3.3 Use Case Summary

ID	Use Case	Primary Actor(s)
UC-01	Register Account	Student
UC-02	Log In	Student, Admin
UC-03	Complete / Edit Profile	Student
UC-04	Upload Supporting Documents	Student
UC-05	Submit Profile for Verification	Student
UC-06	Perform AI Document & Consistency Check	AI Verification Engine

ID	Use Case	Primary Actor(s)
UC-07	Review Flagged Account	Admin
UC-08	Receive Verification / Rejection Notification	Student
UC-09	Generate Eligibility Matches	AI Matching Engine
UC-10	View Matched Scholarships	Student
UC-11	Apply to Scholarship	Student
UC-12	Track Application Status	Student
UC-13	Add / Edit Scholarship Manually	Admin
UC-14	Import Scholarships from Partner Feed	Admin, Scholarship Partner
UC-15	Configure Eligibility Rules	Admin
UC-16	View Platform Analytics	Admin

3.4 Detailed Use-Case Specification — UC-05 Submit Profile for Verification

The following use-case card follows the structured requirement-card format taught in the course (Function / Description / Inputs / Source / Outputs / Destination / Action / Requirements / Pre-condition / Post-condition / Side effects).

Field	Specification
Function	Submit a completed student profile and its supporting documents for AI/admin verification.
Description	Once a Student has filled in all required profile fields and uploaded the required documents, they trigger a review that determines whether their account becomes Verified.
Inputs	personalDetails, nationality, currentLocation, educationLevel, priorInstitution, qualifications, documents[] (passport/ID, transcript, proof of graduation).
Source	Student, via the Student Web Portal.
Outputs	verificationStatus (Verified / Rejected / PendingAdminReview), aiConfidenceScore, reasons[] (if rejected or flagged).
Destination	Profile and VerificationRecord entities in the database; Notification component (email to Student); Admin verification queue (if flagged).
Action	System validates all mandatory fields are present, stores the documents, and enqueues an AI verification job.
Requirements	FR-09, FR-10, FR-11, FR-12 (Section 3.6).
Pre-condition	Student is registered and logged in; all mandatory profile fields are completed; at least one document per required document type has been uploaded.
Post-condition	Profile.verificationStatus is set to Verified, Rejected, or PendingAdminReview; a VerificationRecord is created; the Student has received a notification email.
Side effects	If auto-verified, the Student immediately gains access to the AI Matching Engine (UC-09); if rejected, the Student may edit and resubmit the profile (looping back to UC-03).

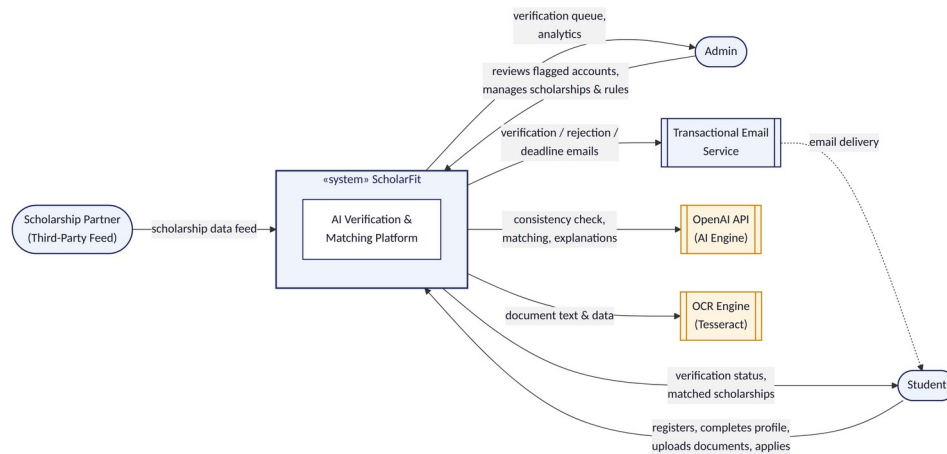


Figure 3.1 — Context Diagram of «system» ScholarFit and its external actors/systems

The context diagram (Figure 3.1) shows ScholarFit as a single «system»-stereotyped box exchanging information with its two human actor types (Student, Admin), one external organisational actor (Scholarship Partner), and three supporting external systems (OpenAI API, OCR engine, transactional email service). It scopes the system boundary before the more detailed interaction, structural and behavioural models in Section 4.

3.5 Functional Requirements

Functional requirements are grouped by module and numbered FR-01 upward, following the “the system shall...” statement style, each with a short rationale in parentheses, as used in the course's own worked examples.

3.5.1 Registration, Authentication & Profile

FR-01 The system shall allow a new user to register a Student account using an email address and password. (*Rationale: baseline access control for every subsequent use case.*)

FR-02 The system shall provide separate, role-based login flows for Student and Admin accounts. (*Rationale: Student and Admin see entirely different dashboards and permissions.*)

FR-03 The system shall allow a Student to complete a profile capturing personal details, nationality, current location, education level, prior institution and qualifications. (*Rationale: these are the exact fields every scholarship's eligibility rules are evaluated against.*)

FR-04 The system shall allow a Student to upload supporting documents, including passport/ID and academic transcripts, in common file formats (PDF, JPG, PNG). (*Rationale: documents are the input to the AI verification engine.*)

FR-05 The system shall allow a Student to edit an incomplete or rejected profile and resubmit it. (*Rationale: rejections must be recoverable, not terminal.*)

3.5.2 AI Verification

FR-06 The system shall extract text and structured data from uploaded documents using OCR before AI review. (*Rationale: raw AI consistency checks are more reliable against structured extracted text than against raw images.*)

FR-07 The system shall compute an AI confidence score reflecting the legitimacy, consistency and completeness of a submitted profile and its documents. (*Rationale: the score is the single gate deciding auto-verify vs. escalate.*)

FR-08 The system shall automatically verify a profile when its AI confidence score is at or above the configured auto-verify threshold. *(Rationale: routine, unambiguous cases should not consume admin time.)*

FR-09 The system shall route a profile to the Admin verification queue when its AI confidence score is below the auto-verify threshold. *(Rationale: human-in-the-loop is required for every ambiguous case (Objective O4).)*

FR-10 The system shall generate a specific, human-readable reason for every rejection or escalation (e.g. "passport expiry page missing"), rather than a generic failure message. *(Rationale: supports Objective O6 (explainable outcomes).)*

FR-11 The system shall notify a Student by email of the outcome of their verification, including the reason if rejected. *(Rationale: students should never be left wondering why their account was not verified.)*

FR-12 The system shall allow an Admin to approve or reject a flagged account, recording the Admin's identity and decision timestamp. *(Rationale: supports the audit-logging requirement in Section 6 (Security & Compliance).)*

FR-13 The system shall flag likely duplicate accounts or reused documents across separate submissions for Admin review. *(Rationale: prevents a single fraudulent applicant from obtaining multiple matches (Section 6 of the product specification).)*

3.5.3 AI Matching & Application

FR-14 The system shall evaluate a Verified student's profile against every active scholarship's eligibility rules and return only the scholarships the student qualifies for. *(Rationale: this is the platform's core value proposition.)*

FR-15 The system shall rank returned scholarships by a computed fit score. *(Rationale: students should see their best-fit opportunities first.)*

FR-16 The system shall allow a Verified student to apply to a matched scholarship directly through the platform. *(Rationale: closes the loop from matching to conversion.)*

FR-17 The system shall allow a Student to track the status of every application they have submitted (Draft, Submitted, Under Review, Accepted, Rejected, Withdrawn). *(Rationale: students need visibility into outcomes they cannot otherwise observe.)*

FR-18 The system shall notify a Student of upcoming scholarship deadlines and any missing requirements for opportunities they are eligible for. *(Rationale: reduces missed opportunities due to deadline oversight.)*

3.5.4 Scholarship & Admin Management

FR-19 The system shall allow an Admin to manually add, edit or deactivate a scholarship record and its eligibility rules. *(Rationale: not every scholarship provider will offer a partner feed.)*

FR-20 The system shall allow an Admin to import scholarship records from a third-party partner feed into the same eligibility-rule structure used for manual entry. *(Rationale: the AI matching engine must evaluate all scholarships uniformly regardless of source.)*

FR-21 The system shall provide an Admin dashboard summarising verification-queue size, match rate and application conversion rate. *(Rationale: supports platform monitoring and Objective O3 (usable dashboards).)*

FR-22 The system shall allow an Admin to suspend or reinstate a Student account. *(Rationale: necessary for handling confirmed fraud or abuse.)*

3.6 Non-Functional Requirements

Non-functional requirements are organised using the course's Product / Organisational / External taxonomy for non-functional requirements (Sommerville, 2016), rather than as an unstructured list.

3.6.1 Product Requirements

Category	Requirement
Efficiency — Performance	NFR-01: 95% of verification decisions (auto-verify or escalate) shall be produced within 24 hours of submission.
Efficiency — Space	NFR-02: Document storage shall use object storage rather than the relational database, keeping database row sizes bounded regardless of document volume.
Usability	NFR-03: A first-time Student shall be able to complete registration and profile submission without external assistance, in a single session of under 20 minutes.
Dependability	NFR-04: The AI verification engine's auto-verify decision shall achieve a false-positive rate (incorrectly auto-verifying an ineligible/fraudulent profile) of under 1%, reviewed monthly against a held-out test set (Section 7.4).
Security	NFR-05: All passport, ID and transcript documents shall be encrypted at rest and in transit, and shall be accessible only to the owning Student and to Admins performing an active review.

3.6.2 Organisational Requirements

Category	Requirement
Environmental	NFR-06: The platform shall run as a web application accessible from standard desktop and mobile browsers, without requiring a native app install.
Operational	NFR-07: The verification and matching workloads shall run as background jobs on a queue, decoupled from the request/response cycle of the web API, so that a slow AI call never blocks the user interface (Section 5).
Development	NFR-08: The codebase shall be TypeScript end-to-end (frontend and backend) to allow a small team to work across the stack without a language-context switch.

3.6.3 External Requirements

Category	Requirement
Regulatory / Legislative	NFR-09: Handling of nationality and identity-document data shall follow applicable personal-data-protection principles for every jurisdiction a student is registered from (Section 6).
Ethical	NFR-10: The AI verification engine's decisions shall be logged and auditable so an Admin (and, on request, the affected Student) can see the basis of a rejection, avoiding unaccountable automated decision-making.
Accounting / Safety-Security	NFR-11: Every Admin verification decision and every account-level status change shall be recorded in an immutable audit log (Section 6).

3.6.4 Non-Functional Requirement Metrics

Each non-functional requirement is made verifiable by attaching it to a measurable metric, following the course's guidance that NFRs must be quantified rather than stated qualitatively.

Property	Metric
Speed	Verification job turnaround time (minutes/hours); API p95 response time (ms).
Size	Average object-storage footprint per student (MB); database row count growth per 1,000 students.
Ease of use	Time to complete registration + profile submission (minutes); support-ticket rate per 100 registrations.
Reliability	AI verification false-positive / false-negative rate (%); job-queue failure rate (%).
Robustness	System behaviour under malformed/corrupted document uploads; time to recover from an AI-provider outage.
Portability	Percentage of functionality available unchanged across supported browsers/devices.

4. System Models

This section presents the four UML system models required by the assessment brief: an Activity Diagram (data-driven, behavioural model), a Sequence Diagram (interaction model), a Class Diagram (structural model) and a State Diagram (event-driven, behavioural model). All four model the same underlying process — student registration through verification to matching and application — from four complementary perspectives, following the course's own framing of Activity/Sequence/Class/State as Behavioural (data-driven and event-driven), Interaction and Structural views respectively.

4.1 Activity Diagram

The activity diagram below models the end-to-end registration-to-application process as a data-driven flow across four swimlanes — Student, ScholarFit Platform, AI Verification Engine and Admin — each lane showing the activities that actor/component is responsible for. It uses the standard notation taught in the course: a filled circle for the start node, rounded rectangles for activities, a diamond for a decision node with guard conditions in square brackets (e.g. [Auto-Verify], [Flag for Review]), a synchronisation bar where the two possible verification paths (auto-verify and admin-approved) rejoin before notification, and a nested circle for the end node.

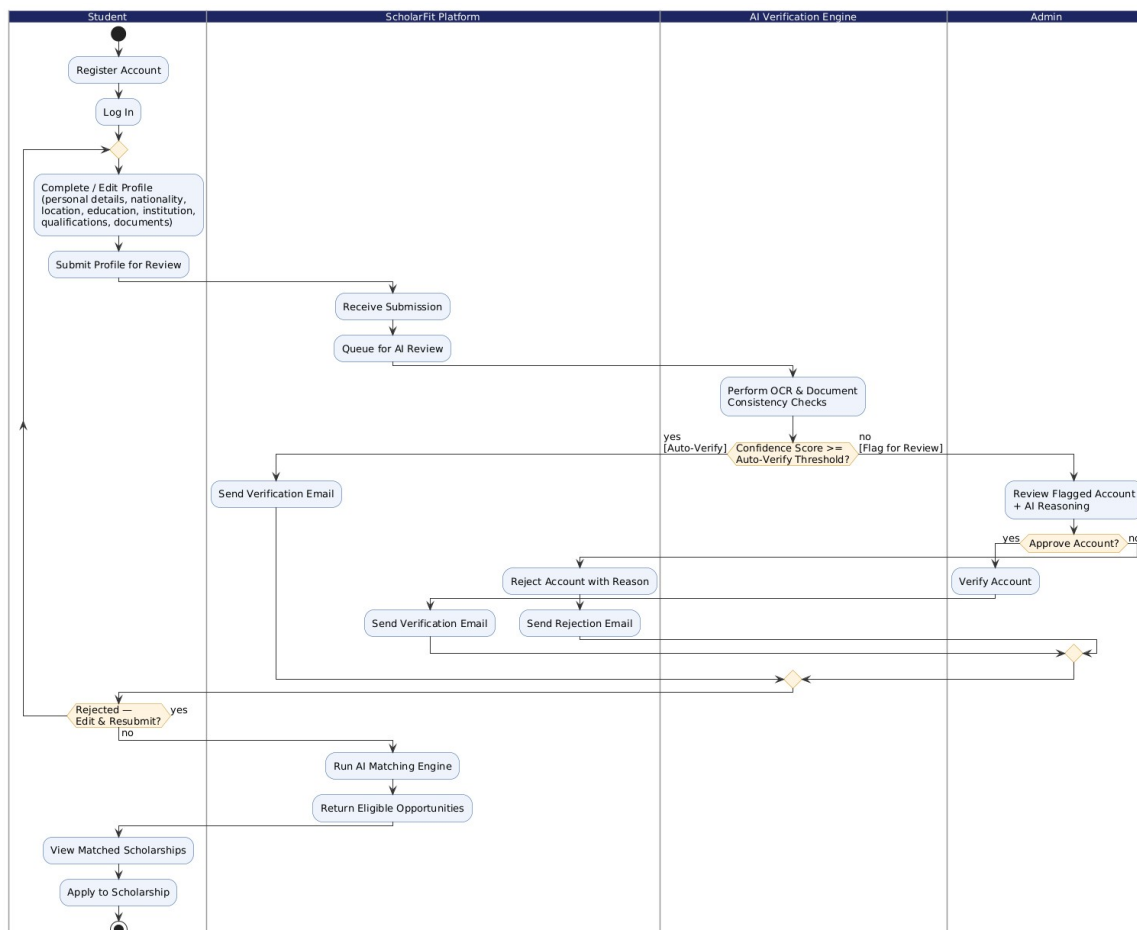


Figure 4.1 — Activity Diagram: Registration → Verification → Matching → Application

Two decision points drive the process: the AI Verification Engine's confidence-threshold check (auto-verify vs. flag for review) and the Admin's approve/reject decision on flagged accounts. A rejected account can loop back to "Edit Profile & Resubmit", re-entering the flow at "Submit Profile for Review"

rather than terminating — this loop is deliberate, since Objective O6 requires rejections to be explainable and recoverable rather than final.

4.2 Sequence Diagram

The sequence diagram models the interaction for use case UC-05/UC-06 (Submit Profile for Verification / Perform AI Document & Consistency Check), the core differentiator of the platform. Following the interaction-model notation taught in the course, each participant (Student, Web App, ScholarFit API, AI Verification Engine, Database, Admin, Email Service) is shown with a lifeline, activation bars appear implicitly on the message-receiving side of synchronous calls, and an alt combined fragment (matching the course's own "alt [condition]" convention) separates the auto-verify branch from the admin-escalation branch, with a nested alt further separating the Admin's approve and reject outcomes.

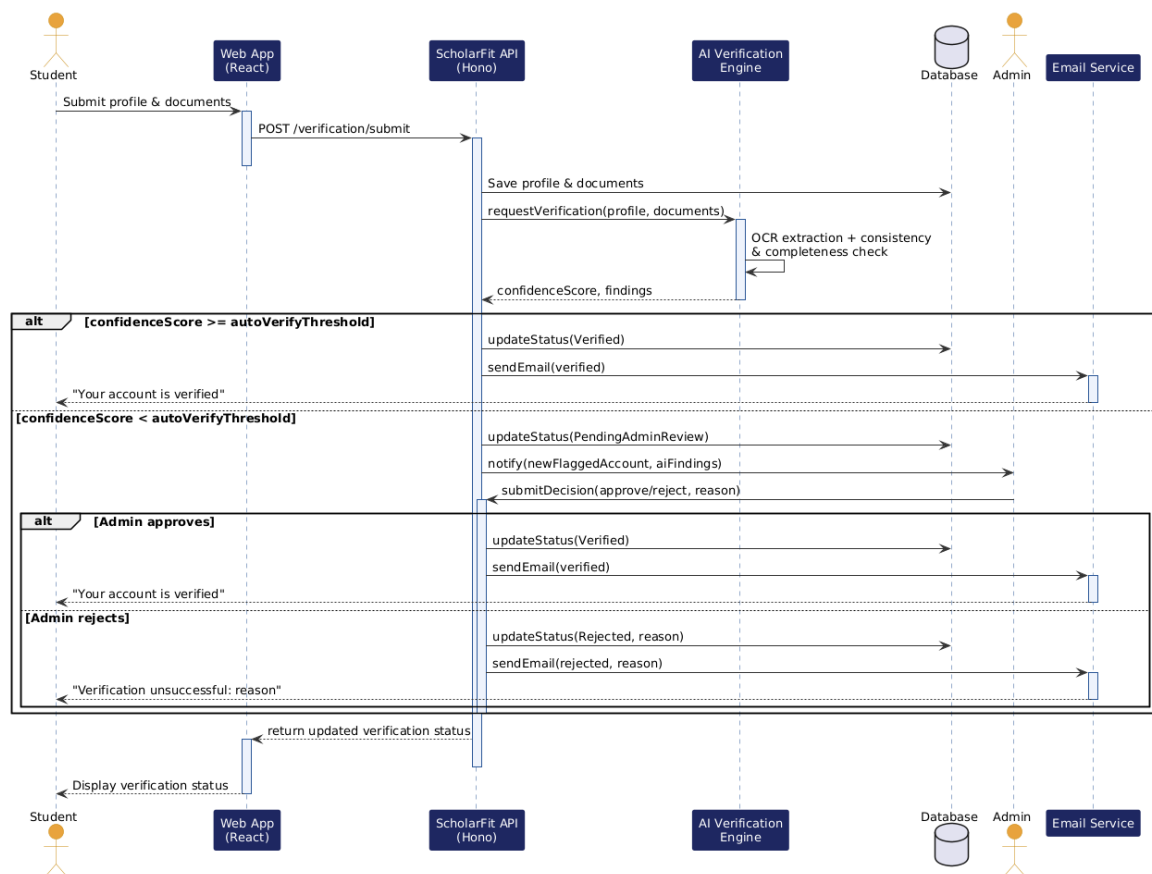


Figure 4.2 — Sequence Diagram: Submit Profile for Verification

This diagram traces directly to functional requirements FR-06 through FR-12: the OCR/consistency check (FR-06, FR-07), the auto-verify gate (FR-08), the escalation path (FR-09), the explainable-rejection reason attached to the outcome (FR-10), and the outcome notification (FR-11). It is also the basis for the use-case-based system test cases described in Section 7.3.

4.3 Class Diagram

The class diagram models the structural view of ScholarFit's core domain. User is generalised into Student and Admin, matching the course's generalisation notation (open arrowhead pointing to the more general superclass). A Student owns exactly one Profile (association, multiplicity 1-1); a Profile aggregates one or more Documents (aggregation, shown with a diamond at the "whole" end) and

generates one or more VerificationRecord entries, each of which an Admin may review. A Scholarship aggregates one or more EligibilityRule entries — the structure that both the AI Matching Engine and the Admin's manual scholarship-entry form must share (Objective O7) — and produces zero or more Match records against Students, each of which may lead to at most one Application.

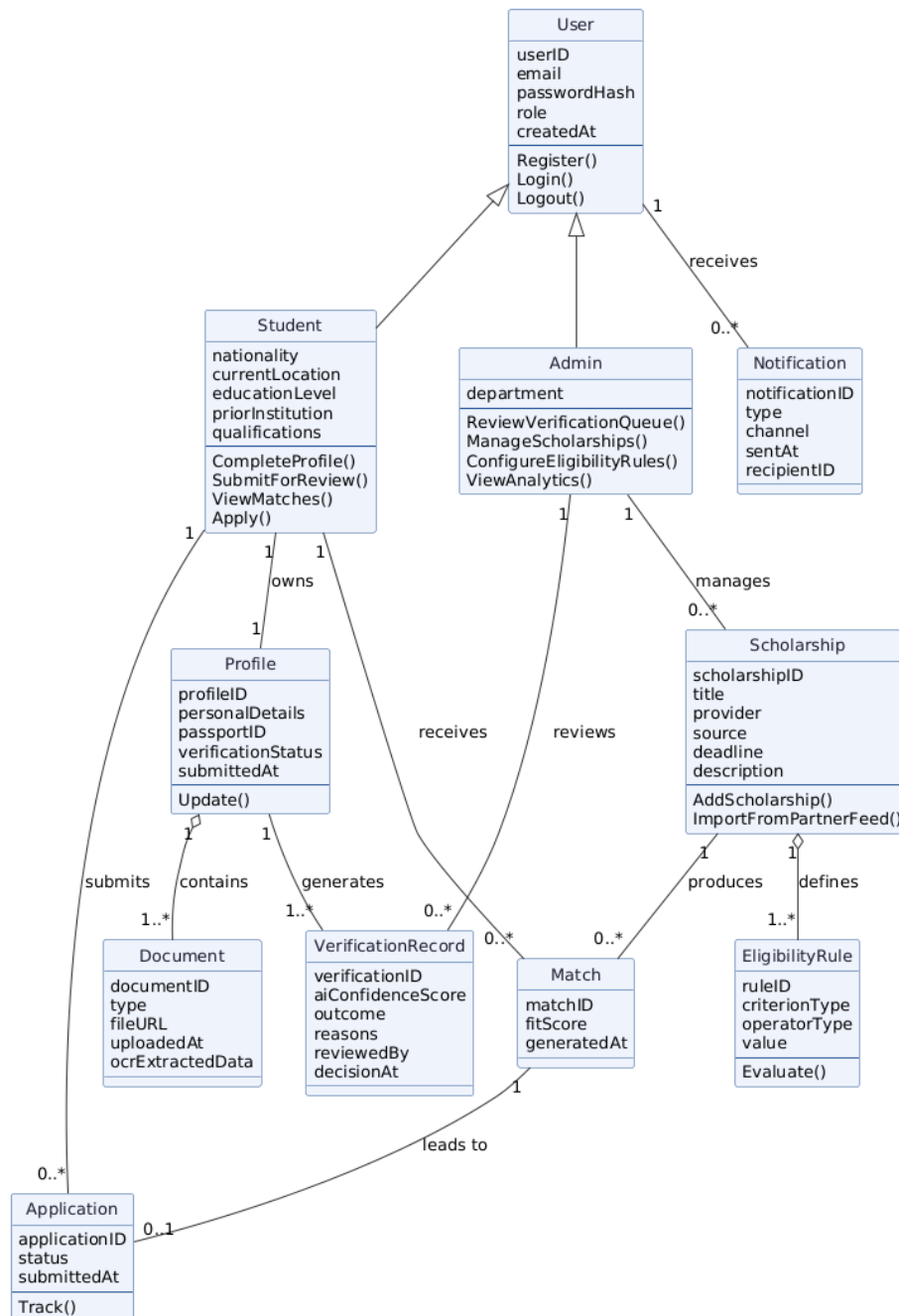


Figure 4.3 — Class Diagram: Core Domain Model

Attributes and operations are shown in the three-compartment class-box notation used throughout the course (class name / attributes / operations). Associations are named to indicate the relationship they represent (owns, contains, generates, reviews, manages, defines, receives, produces, leads to, submits), and multiplicities are shown at each association end, matching the course's own MHC-PMS-style class diagrams.

4.4 State Diagram

The state diagram models the event-driven behaviour of a Student's verification status — the single state machine most central to the platform's value proposition. States are shown as rounded rectangles (optionally annotated with a do/ internal activity, as in the course's microwave-oven example), transitions are labelled arrows triggered by named events, and initial/final states use the standard filled-circle and ringed-circle notation.

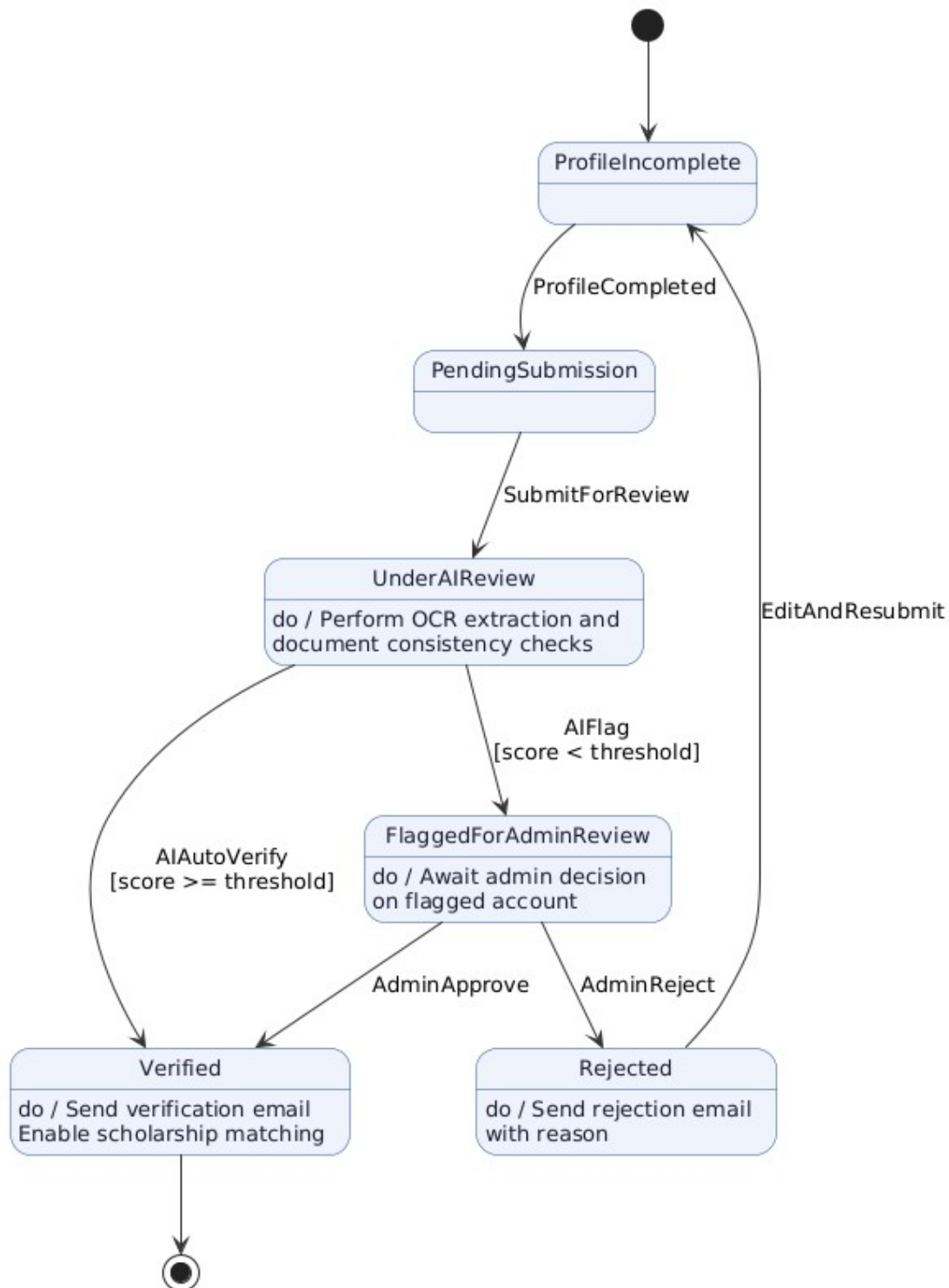


Figure 4.4 — State Diagram: Student Verification Status

Two supporting tables document the state machine in the two-table style used in the course's own worked example (state → description, and stimulus/event → description).

State	Description
ProfileIncomplete	Initial state after registration; the Student has not yet completed all mandatory profile fields.
PendingSubmission	All mandatory fields are complete, but the profile has not yet been submitted for review.
UnderAIReview	The AI Verification Engine is running OCR extraction and consistency/completeness checks.
Verified	The account has passed verification (auto or admin-approved) and can access the AI Matching Engine.
FlaggedForAdminReview	The AI confidence score was below the auto-verify threshold; the case is in the Admin's verification queue.
Rejected	The Admin (or, in edge cases, the AI directly) rejected the submission; a specific reason has been generated and emailed.
Event / Stimulus	Description
ProfileCompleted	The Student has filled in every mandatory profile field and uploaded every required document type.
SubmitForReview	The Student explicitly submits the completed profile for verification.
AIAutoVerify	The AI Verification Engine's confidence score is at or above the auto-verify threshold.
AIFlag	The AI Verification Engine's confidence score is below the auto-verify threshold.
AdminApprove	An Admin reviews a flagged account and approves it.
AdminReject	An Admin reviews a flagged account and rejects it, recording a reason.
EditAndResubmit	The Student edits a rejected profile and re-enters the ProfileIncomplete state to try again.

5. System Architecture Design

Architectural design is "the process of identifying the sub-systems making up a system and the framework for sub-system control and communication" (Sommerville, 2016). Before selecting a pattern, the course's eight architectural design decision questions are answered directly for ScholarFit:

Design Decision	Answer for ScholarFit
Is there a generic application architecture template?	Yes — an information-system / transaction-processing template, since ScholarFit manages structured records (profiles, documents, scholarships) through create/verify/match/apply transactions.
How will the system be distributed across processors/hardware?	A single web-tier + API-tier + background-worker-tier deployment (Section 5.3), with the database and object storage as separate managed services.
What architectural pattern(s) should be used?	Layered architecture for the overall structure, client-server for deployment, and an MVC-influenced organisation within the frontend (Section 5.2).
What is the fundamental structuring approach?	Separation of the presentation, application/business-logic, AI-processing and data layers, each depending only on the layer beneath it.
How should the system be decomposed into sub-components?	By business capability (Auth, Profile, Verification, Matching, Scholarship, Application, Notification), detailed as components in Section 6.
What control strategy should be used?	Centralised control through the Hono API for request/response flows, and an asynchronous job queue for the non-deterministic, potentially slow AI verification/matching work.
How will the architecture satisfy non-functional requirements?	See the criteria applied below (performance, security, availability, maintainability).
How should the architecture be documented?	As a layered block diagram (Figure 5.1) supplemented by the component diagram in Section 6, following the course's own "simple, informal block diagram" convention for architecture documentation.

5.1 Architectural Pattern Selection and Justification

The course teaches that architecture should be driven by non-functional requirements, offering explicit criteria for performance, security, safety, availability and maintainability (Sommerville, 2016). Applying those criteria to ScholarFit's own non-functional requirements (Section 3.6) leads to the following choice:

- **Layered (tiered) architecture** is adopted as the primary structuring pattern. The course's security criterion — "use a layered architecture, with critical assets placed in the innermost layers" — maps directly onto NFR-05 (documents encrypted and access-restricted): sensitive documents and verification records sit in the Data Layer, reachable only through the Application Layer's access-controlled services, never directly from the Presentation Layer. The course's maintainability criterion — "use fine-grain, replaceable components" — supports Objective O7, letting the AI Verification and Matching workers evolve independently of the rest of the system.
- **Client-server** describes the deployment relationship between the Presentation Layer (Student and Admin web clients) and the Application Layer (the Hono API on Bun), connected over HTTPS — the course's textbook description of client-server ("a set of stand-

alone servers... offering services... a set of clients... a network") applies directly, since students and admins access the same backend from many independent browser sessions.

- **MVC-influenced frontend organisation** is used within each web client: the React component tree acts as the View, application/query state as the Model, and route/action handlers as the Controller. The course notes MVC "serves as a basis of interaction management in many web-based systems" and is well supported by modern frontend frameworks, which matches the React + TypeScript stack fixed for this project.

Repository architecture and pipe-and-filter were both considered and rejected as the primary pattern: repository architecture would suit a system centred on one shared dataset accessed uniformly by many tools (e.g. a CAD system), which does not reflect ScholarFit's request/response web-service nature; pipe-and-filter suits batch data-transformation pipelines and is explicitly unsuitable for interactive systems. Elements of both nonetheless appear inside the architecture: the PostgreSQL database plays a repository-like role for the AI workers and the API, and the document-verification job (OCR → consistency check → decision) is internally a short pipe-and-filter chain.

5.2 Architecture Diagram

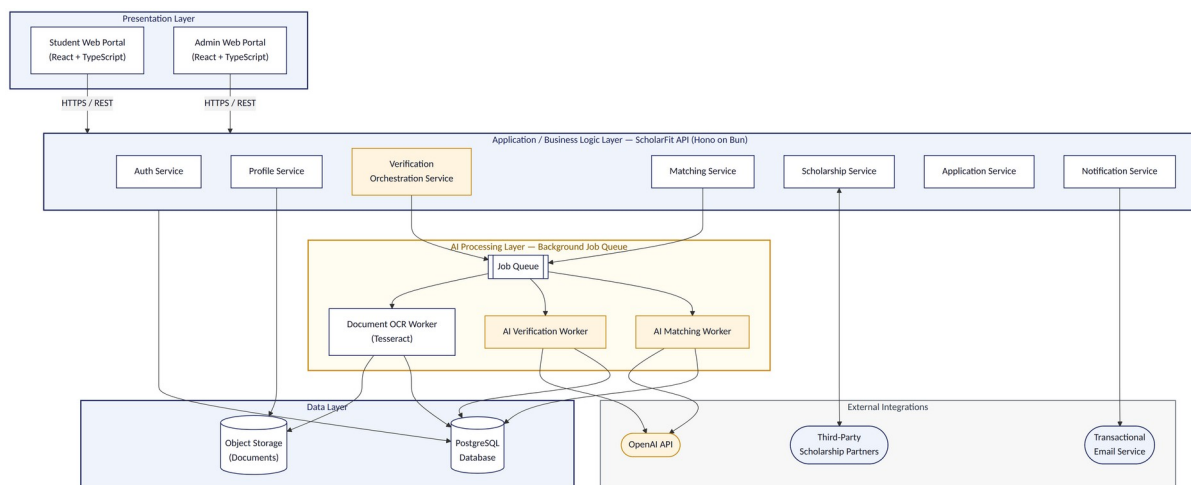


Figure 5.1 — Layered System Architecture of ScholarFit

The architecture is organised into four layers plus an external-integration boundary:

- **Presentation Layer** — the Student Web Portal and Admin Web Portal, both built in React + TypeScript, communicating with the Application Layer over HTTPS/REST.
- **Application / Business Logic Layer** — the ScholarFit API, built on Hono running on Bun, decomposed into Auth, Profile, Verification Orchestration, Matching, Scholarship, Application and Notification services (detailed as components in Section 6). This layer owns all business rules and is the only layer permitted to read or write the Data Layer directly on behalf of a user request.
- **AI Processing Layer** — a background job queue consumed by three workers (Document OCR Worker, AI Verification Worker, AI Matching Worker). Placing AI work behind a queue, rather than inline in the request/response cycle, satisfies NFR-07 (a slow AI call must never block the UI) and Objective O8 (efficient, rate-limited use of external AI resources).
- **Data Layer** — a PostgreSQL database (chosen and justified in Section 5.3.1) for all structured, relational records, and object storage for the documents themselves, keeping large binary content out of the relational database (NFR-02).

- **External Integrations** — the OpenAI API (AI consistency checks, matching explanations), third-party scholarship partner feeds, and a transactional email service, each accessed only from the layer that owns the relevant responsibility (AI workers, Scholarship service, Notification service respectively).

5.3 Supporting Design Decisions

5.3.1 Database Choice

A relational database (PostgreSQL) is proposed for ScholarFit's Data Layer. The core domain — students, profiles, documents, scholarships and, most importantly, eligibility rules that must be evaluated consistently by the AI Matching Engine — is highly structured and relationship-heavy (Section 4.3), which relational schemas and foreign-key constraints model naturally and enforce automatically. PostgreSQL specifically also offers native JSON/JSONB columns, which allows the semi-structured, criterion-by-criterion shape of an EligibilityRule (Section 6) to be stored flexibly without abandoning relational integrity for the rest of the schema.

ASSUMPTION: PostgreSQL is proposed as the concrete relational database product; the product specification leaves the exact database "to be confirmed" and this choice should be re-validated against the founder's hosting/cost constraints before Sprint 0 (Section 8).

5.3.2 Authentication, Authorization and Session Handling

The Auth component (Section 6) issues a signed session token on login, carrying the user's role (Student or Admin). Every API route in the Application Layer checks the token's role claim before executing, so that, for example, only an Admin-role token can reach the verification-queue or scholarship-management endpoints (FR-02, NFR-05). Role-based access control is chosen over a more granular permission system because the product specification defines exactly two roles with materially different capabilities, and a simple role check is easier to audit correctly than a fine-grained permission matrix — directly serving the maintainability and security criteria used to justify the architecture above.

6. Component-Based Model

Component-Based Software Engineering (CBSE) is "the process of defining, implementing and integrating or composing loosely coupled, independent components into systems" (Sommerville, 2016), where a component is a modular, deployable and replaceable part of a system that encapsulates its implementation and exposes its functionality only through a set of interfaces. ScholarFit is decomposed into eight independent components, each independently specified by a Provides interface (the services it offers) and one or more Requires interfaces (the services it depends on), following the course's provides/requires interface vocabulary.

6.1 Component Catalogue

Component	Provides Interface	Requires Interface(s)
Auth Component	IAuthentication	— (no internal dependency; foundational component)
Profile Management Component	IProfileManagement	IAuthentication
Document Verification Component	IVerification	IProfileManagement, IOCRExtraction, IAConsistencyCheck, INotification
Matching Engine Component	IMatching	IProfileManagement, IScholarshipCatalog, IAIMatchExplanation
Scholarship Component	IScholarshipCatalog	IPartnerFeedData
Application Tracking Component	IApplicationTracking	IMatching, IProfileManagement, INotification
Notification Component	INotification	IEmailDelivery
Admin Dashboard Component	IAdminOperations	IVerification, IScholarshipCatalog, IProfileManagement

Four further services are acquired rather than built in-house, following the "component as a service" pattern the course notes is increasingly replacing formal component-model standards such as EJB, COM or CCM: the OCR Service (Tesseract), the OpenAI Service, the Email Gateway, and the Partner Feed Adapter. Each is treated as a component that exposes only a Provides interface, since ScholarFit consumes but never implements them.

6.2 Component Diagram

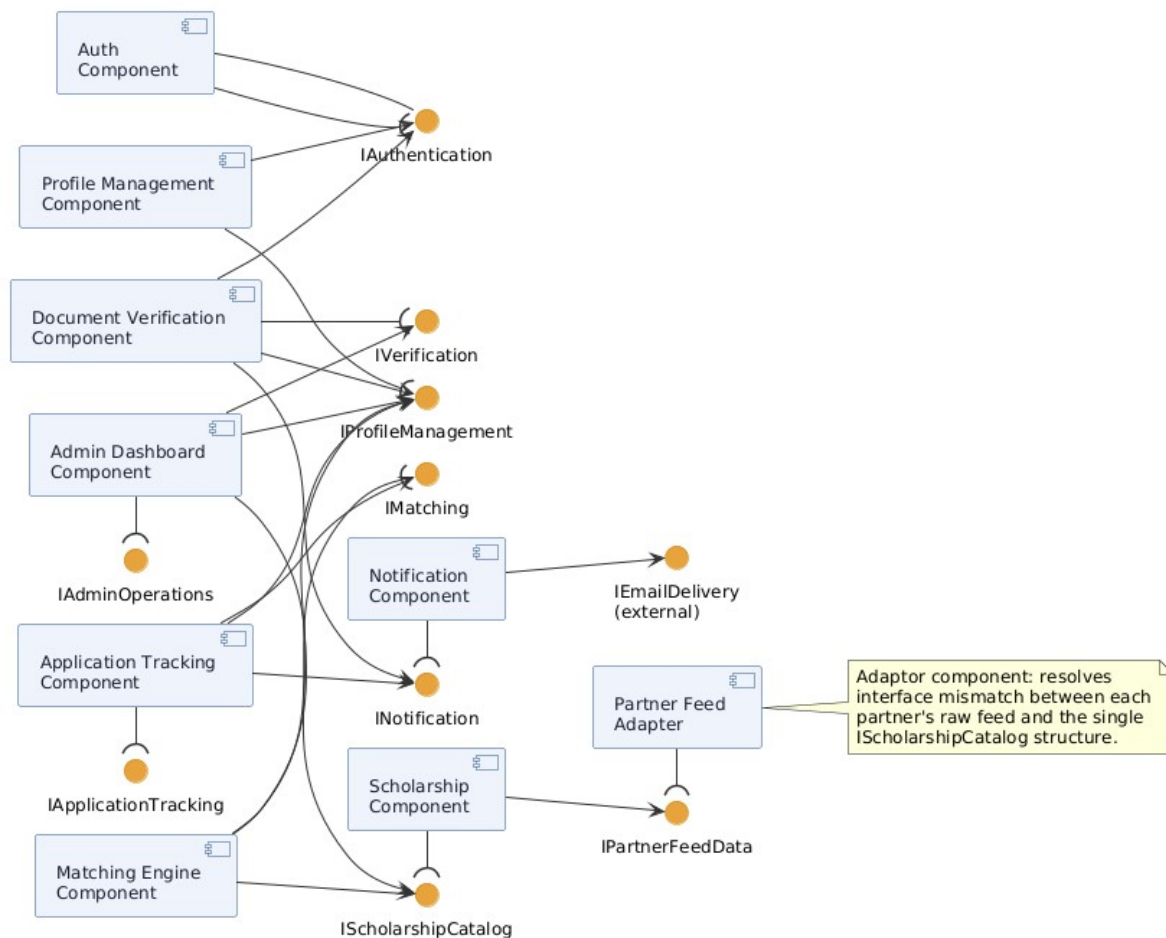


Figure 6.1 — Component-Based Model of ScholarFit, showing Provides/Requires interfaces

Reading the diagram top-down: the two "tier-1" components (Admin Dashboard, Application Tracking) sit closest to the user interface and depend on the "tier-2" business-capability components (Document Verification, Matching Engine, Scholarship); those in turn depend on the shared "tier-3" services (Profile Management, Auth, Notification); and the bottom tier holds the four acquired/external components. No arrow skips more than one tier, which keeps the coupling shallow and each component's role non-overlapping — the separation-of-concerns principle the course states explicitly as the guiding trade-off in component composition.

6.3 Composition Types

The course identifies three ways components can be composed — sequential, hierarchical and additive. ScholarFit's composition is predominantly hierarchical: a higher-tier component calls directly on a lower-tier component's provided interface (for example, Matching Engine Component calls Scholarship Component's IScholarshipCatalog), so the provided interface of the callee must match the required interface of the caller exactly, as the course specifies.

One composition is explicitly additive/adaptive rather than purely hierarchical: the Partner Feed Adapter is an adaptor component in the course's own sense — it exists specifically to resolve the interface incompatibility between each scholarship partner's raw, inconsistent data feed and the single IScholarshipCatalog / EligibilityRule structure the rest of the system expects (Section 3.5.4, FR-20; and the AI-assisted "scholarship data extraction" capability from the product specification).

Without this adaptor, every new partner integration would otherwise require changing the Scholarship Component itself, which would violate the independence and replaceability principles CBSE is built on.

6.4 Reusability Characteristics

The course lists five characteristics a component needs to be genuinely reusable: composable, deployable, documented, independent and standardised. The table below assesses each ScholarFit component against these characteristics.

Component	Composable	Deployable	Independent	Notes
Auth Component	Yes — only via IAuthentication	Yes — stateless service	Yes	No other component depends on its internals, only its interface.
Profile Management	Yes	Yes	Partially	Depends on Auth's interface, but not its implementation.
Document Verification	Yes	Yes — can scale as a separate worker	Yes	Can be replaced with a different OCR/AI vendor without changing callers, provided IVerification is preserved.
Matching Engine	Yes	Yes — can scale as a separate worker	Yes	Matching algorithm can evolve (e.g. rule-based to ML-ranked) behind the same interface.
Scholarship	Yes	Yes	Yes	Isolates all partner-feed variability behind the Partner Feed Adapter.
Application Tracking	Yes	Yes	Partially	Depends on Matching and Profile Management interfaces.
Notification	Yes — only via INotification	Yes	Yes	Email provider can be swapped by replacing only the Email Gateway.
Admin Dashboard	Yes	Yes	Partially	Aggregates several other components' interfaces; expected for a dashboard.

7. System Testing Strategy

The course frames testing as three sequential stages — development testing, release testing and user testing — rather than the generic four-level unit/integration/system/acceptance scheme (Sommerville, 2016). ScholarFit's testing strategy follows this pipeline directly, with particular attention to the fact that the AI verification and matching engines are not fully deterministic, so conventional pass/fail testing alone is not sufficient (Objective O10).

7.1 Testing Objectives and Expected Outcomes

Before selecting stages and techniques, the strategy states what the testing effort is for. Each testing objective below is paired with the method that serves it and the concrete, observable outcome that indicates the objective has been met, so that testing success is judged against pre-agreed evidence rather than a subjective sense of "enough testing".

Testing Objective	Primary Method	Expected Outcome (Exit Criterion)
TO-1 — Prove every functional requirement is implemented as specified.	Requirements-based testing (7.3), traced FR-by-FR.	Every requirement FR-01–FR-22 has at least one passing test case derived from it; the sample documentation format in 7.5 records the input, expected result and outcome for each.
TO-2 — Prove the components integrate through their declared interfaces only.	Component testing against the Provides/Requires interfaces in Section 6.	Every interface in the Section 6 catalogue is exercised by a passing test; no test requires reaching into another component's internals.
TO-3 — Prove the end-to-end student journey works as modelled.	System testing driven by the Figure 4.2 sequence diagram and Figure 4.1 activity diagram.	All three verification branches (auto-verify, admin-approve, admin-reject) complete successfully, and the state transitions observed match Figure 4.4 exactly.
TO-4 — Establish that the non-deterministic AI core behaves within agreed tolerances.	Golden-dataset regression and confidence-threshold tuning (7.4).	Agreement with the golden dataset is at or above the agreed release threshold, and the measured false-positive rate satisfies NFR-04 (< 1%).
TO-5 — Confirm sensitive documents are protected as required.	Security-focused testing of access control on document and verification-record endpoints (NFR-05).	No test succeeds in retrieving another student's document or verification record; every Admin decision appears in the audit log (NFR-11).
TO-6 — Confirm the system is acceptable to its real users.	User testing — Alpha, Beta and Acceptance (7.2, 7.7).	A first-time student completes registration and profile submission unaided in under 20 minutes (NFR-03), and the product owner signs off the acceptance checklist.
TO-7 — Prevent regression as the system grows.	Automated regression suite executed in CI on every change (7.3).	The full suite runs on every commit and blocks any merge that reduces the passing-test count or golden-dataset agreement.

7.2 Testing Pipeline

Stage	Sub-stage	What is Tested for ScholarFit
Development	Unit testing	Individual functions/methods: eligibility-rule evaluator logic,

Stage	Sub-stage	What is Tested for ScholarFit
Testing		confidence-score threshold gate, fit-score calculation, form validation.
Development Testing	Component testing	Interfaces between components (Section 6): e.g. Matching Engine Component correctly consumes Scholarship Component's IScholarshipCatalog; Notification Component correctly consumes the Email Gateway's IEmailDelivery.
Development Testing	System testing	The integrated system's component interactions: the full registration → verification → matching → application chain, exercised through the API.
Release Testing	—	A separate testing pass on a complete release candidate, treating ScholarFit as a black box, to confirm the system is good enough for staging/production release.
User Testing	Alpha	Internal team members role-play Student and Admin journeys at the development site before any external exposure.
User Testing	Beta	A limited cohort of real students and partner-organisation admins use a pre-production release and report issues.
User Testing	Acceptance	The project stakeholder (product owner) formally tests the system against the requirements in Section 3 before sign-off (Section 7.7).

7.3 Testing Techniques

- **Equivalence partitioning.** Student nationality, education-level and document-type inputs are partitioned into groups expected to be processed identically by the eligibility engine (e.g. "nationality present in an eligibility rule's allow-list" vs. "nationality absent"), and one representative test case is drawn from each partition rather than testing every possible value.
- **Guideline-based / boundary testing.** Tests are derived from common-error checklists: an empty document upload, an oversized file, a passport image with an unreadable/cropped expiry page, a profile with every optional field but no mandatory field, and a confidence score exactly at the auto-verify threshold boundary.
- **Requirements-based testing.** Test cases are derived systematically from individual functional requirements, following the course's own worked derivation. For FR-08 ("the system shall automatically verify a profile when its AI confidence score is at or above the configured auto-verify threshold"), five related test cases are produced: score just above threshold (auto-verify), score exactly at threshold (auto-verify, boundary), score just below threshold (escalate), score at zero (escalate with maximum-severity reasons), and a missing/failed OCR extraction (escalate with a specific "document unreadable" reason).
- **Scenario testing.** Realistic narrative scenarios are written and used to derive test cases, in the course's own style — for example: "A Malaysian student uploads a blurry photograph of her passport's information page; the OCR extraction returns a low-confidence name field; the AI Verification Engine should flag the account for Admin review with the specific reason 'passport name field low OCR confidence', not a generic rejection."
- **Use-case / sequence-diagram-based testing.** The sequence diagram in Figure 4.2 is used directly as the basis for system test design: one test traverses the auto-verify branch, one traverses the admin-approve branch, and one traverses the admin-reject branch, confirming every message in the diagram occurs in the correct order with the correct payload.

- **Test-driven development (for deterministic backend logic).** Business-rule-heavy code such as the eligibility-rule evaluator and the fit-score calculator is developed test-first: write a failing test, implement the minimum code to pass it, then refactor — chosen specifically for this logic because, unlike the AI calls, its expected output is fully deterministic and therefore suits TDD.
- **Regression testing.** The full test suite is re-run in CI on every change (Section 8), to confirm that a change to one component (e.g. the Scholarship Component's partner-feed adapter) has not broken another (e.g. the Matching Engine Component).

7.4 Testing the Non-Deterministic AI Pipeline

Because the AI Verification and Matching engines call an external large-language-model API, their outputs are not guaranteed to be identical on every run, which conventional exact-match assertions cannot handle. ScholarFit's strategy for this is:

- **Golden-dataset regression.** A fixed set of representative student profiles and documents, each with a known expected outcome (Verified / Rejected / Escalated, with an expected reason category), is re-run against the AI Verification Engine on every release; a drop in agreement with the golden set below an agreed threshold blocks release.
- **Confidence-threshold tuning tests.** The auto-verify threshold (FR-08) is tuned and re-validated against the golden dataset's known false-positive/false-negative rates (NFR-04), rather than fixed arbitrarily.
- **Human-in-the-loop spot checks.** A random sample of auto-verified accounts is periodically re-reviewed by an Admin, closing the loop on any silent drift in the AI's behaviour.
- **Fairness/bias testing.** The golden dataset deliberately includes profiles across a range of nationalities, education systems and document qualities, to check the verification engine does not systematically disadvantage any particular group.

7.5 Sample Test Case Documentation

The course does not prescribe a specific ISTQB-style column set; instead it defines four testing artefacts — test cases, test data, test results and test reports (Sommerville, 2016) — which the table below is built around directly.

ID	Test Case (spec of input & expected result)	Test Data	Test Result / Report
TC-01	Profile with confidence score above threshold is auto-verified (FR-08).	Complete profile; clear passport scan; score = 0.94, threshold = 0.85.	Pass — status = Verified, no Admin action required.
TC-02	Profile with confidence score below threshold is escalated with a specific reason (FR-09, FR-10).	Profile with a cropped passport image; score = 0.41.	Pass — status = PendingAdminReview, reason = 'passport expiry page missing'.
TC-03	Admin rejection triggers a reason-specific email (FR-10, FR-11).	Flagged account; Admin rejects with reason 'mismatched graduation date'.	Pass — Student receives email containing the exact reason string.
TC-04	Verified student sees only scholarships they qualify for (FR-14).	Verified profile: Malaysian, Bachelor's level; 10 scholarships in catalogue, 3	Pass — exactly 3 matches returned, ranked by fit score.

ID	Test Case (spec of input & expected result)	Test Data	Test Result / Report
		matching eligibility rules.	
TC-05	Duplicate document reuse across two accounts is flagged (FR-13).	Two Student accounts uploading an identical transcript file hash.	Pass — both accounts flagged for Admin review with reason 'duplicate document detected'.

7.6 Quality Attributes Targeted by the Test Strategy

Software quality is treated as "fitness for purpose" rather than strict specification conformance, since some attributes (e.g. maintainability) cannot be verified by a single test run (Sommerville, 2016). Of the fifteen quality attributes the course lists, the test strategy above deliberately prioritises five, matching them to the Quality Plan structure's "quality goals" section:

Quality Attribute	How the Test Strategy Targets It
Reliability	Golden-dataset regression and requirements-based test derivation (Sections 7.3, 7.4).
Security	Access-control and audit-log checks are part of component testing for the Auth and Admin Dashboard components (NFR-05, NFR-11).
Usability	Alpha/beta user testing directly observes real Student and Admin task completion (Section 7.2).
Efficiency	Performance/load testing of the job queue and API against NFR-01 (24-hour verification turnaround).
Testability	Componentisation (Section 6) with well-defined interfaces keeps each component's component-level tests independent of the others' internals.

7.7 Acceptance Testing Process

Formal acceptance testing follows the six-stage process taught in the course: (1) define acceptance criteria directly from the functional and non-functional requirements in Section 3; (2) plan the acceptance testing schedule against the Sprint 7 window in Section 8; (3) derive acceptance tests from the use cases in Section 3.3 and the sample test cases in Section 7.5; (4) run the acceptance tests with the project stakeholder present; (5) negotiate the treatment of any failed test (fix now vs. accept as a known limitation for a later phase); and (6) accept or reject the system for release on that basis.

8. Gantt Chart

ASSUMPTION: The course material most heavily emphasises Agile/Scrum as the applicable process model for small/medium products (Section 2 course research). The Gantt chart below therefore represents the proposed post-report development schedule for building ScholarFit as a sequence of eight 2-week Agile sprints, starting from a nominal kickoff shortly after this report's own 16 September 2026 submission deadline. Sprint dates are illustrative and should be re-baselined once a real kickoff date and team size are confirmed.

Sprint sequencing follows the dependency logic of the system itself: registration, authentication and profile management must exist before there is anything to verify; the AI verification engine must exist, together with the Admin verification queue (so every AI-flagged case has a human path from day one), before the AI matching engine has any verified profiles to match against; and manual scholarship entry is built before third-party partner-feed ingestion, since manual entry alone is sufficient to validate the matching engine end-to-end.

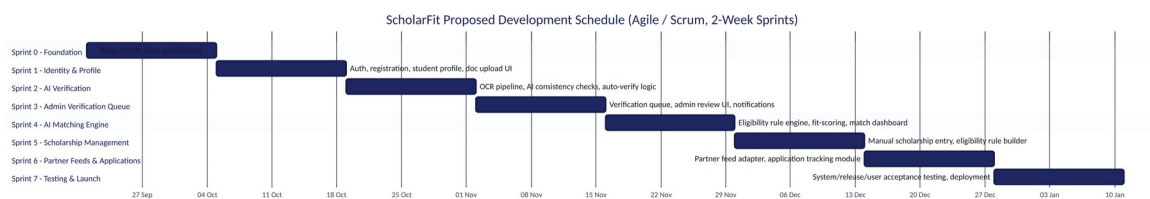


Figure 8.1 — Proposed Development Schedule (Gantt Chart)

Sprint	Goal	Demoable Outcome
Sprint 0 — Foundation	Repository, CI/CD pipeline, environments, base layered architecture (Section 5) scaffolded.	A deployed "hello world" API and web client on staging.
Sprint 1 — Identity & Profile	Auth Component, registration, login, Student profile form, document upload (FR-01–FR-04).	A Student can register, log in and submit a complete profile with documents.
Sprint 2 — AI Verification	OCR pipeline, AI consistency checks, auto-verify threshold logic (FR-06–FR-08).	A submitted profile is automatically verified or held for review.
Sprint 3 — Admin Verification Queue	Verification queue, Admin review UI, outcome notifications (FR-09–FR-12).	An Admin can review a flagged account and the Student receives an outcome email.
Sprint 4 — AI Matching Engine	Eligibility-rule engine, fit-scoring, Student match dashboard (FR-14, FR-15).	A Verified Student sees a ranked list of scholarships they qualify for.
Sprint 5 — Scholarship Management	Manual scholarship entry and eligibility-rule builder for Admins (FR-19).	An Admin can add a new scholarship and see it appear in matching results.
Sprint 6 — Partner Feeds & Applications	Partner Feed Adapter, application-tracking module (FR-16, FR-17, FR-20).	A Student applies to a matched scholarship and tracks its status.
Sprint 7 — Testing & Launch	System, release and user acceptance testing (Section 7); production deployment.	A signed-off, production-deployed ScholarFit ready for its first real cohort of students.

References

- Arsat, N. (2026). CCC2143 Software Engineering [Unpublished lecture notes, Chapters 1–10]. Albukhary International University.
- Boehm, B. W. (1988). A spiral model of software development and enhancement. *Computer*, 21(5), 61–72. <https://doi.org/10.1109/2.59>
- Jacobson, I., Christerson, M., Jonsson, P., & Övergaard, G. (1992). *Object-oriented software engineering: A use case driven approach*. Addison-Wesley.
- Kruchten, P. (2003). *The rational unified process: An introduction* (3rd ed.). Addison-Wesley.
- OpenAI. (2025). OpenAI API documentation. <https://platform.openai.com/docs>
- Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. Scrum.org. <https://scrumguides.org>
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

ASSUMPTION: *The course lecture slides are cited above as an unpublished-course-material entry per APA 7th edition guidance, since no published edition/ISBN was provided with the uploaded files; the terminology and worked examples throughout this report (e.g. the MHC-PMS running example, the provides/requires component vocabulary) are drawn from Sommerville's Software Engineering, which the slides closely follow — please confirm the exact edition your lecturer used so this citation can be finalised.*